VGA CONTROLLER DOCUMENTATION

Digital System Design First Year Project

Made by Ariana-Maria Neagoie and Teodora Popa

Group: 30415

Date: May 2025



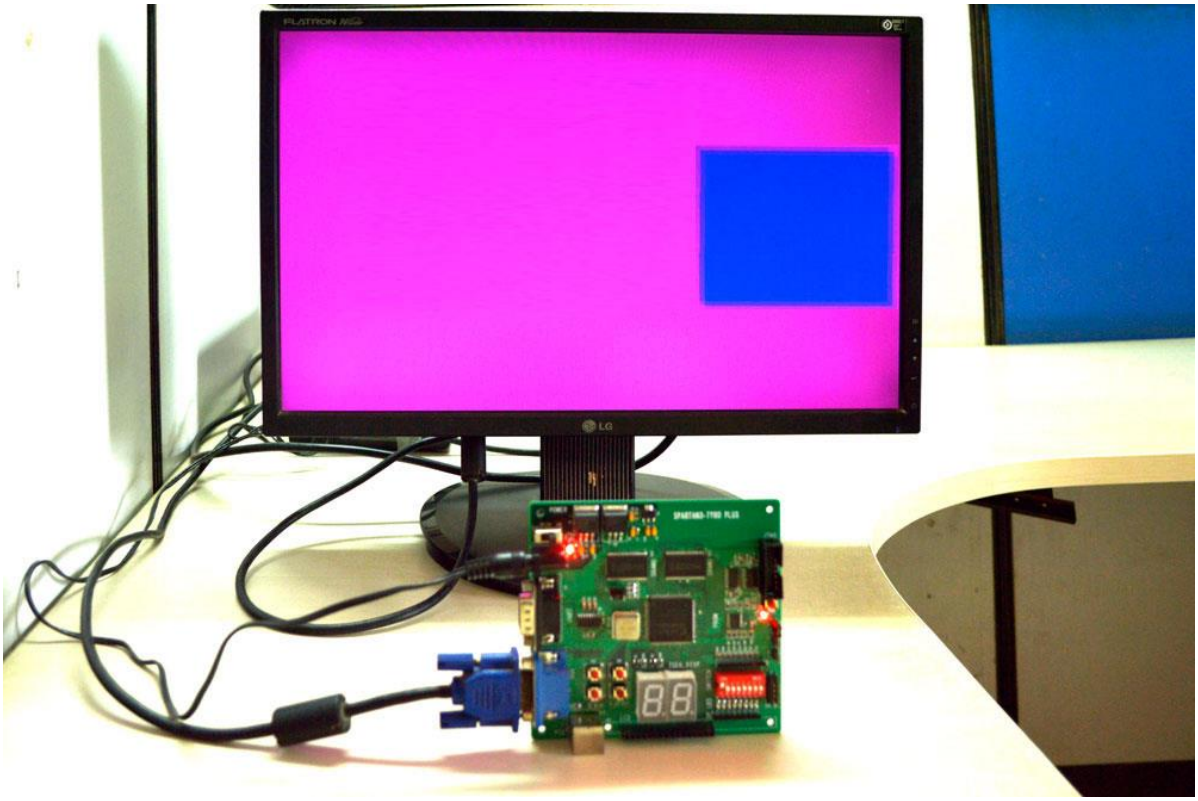
UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

CONTENTS:

1. Task
2. Design
    a. Black Box
    b. Control Unit and Execution Unit
    c. State diagram
    d. Frequency divider
    e. Debouncer
    f. Block Diagram for the first breakdown
    g. Detailed Diagram of the Project
3. User Manual
4. Technical Justifications for the Design
5. Future Developments

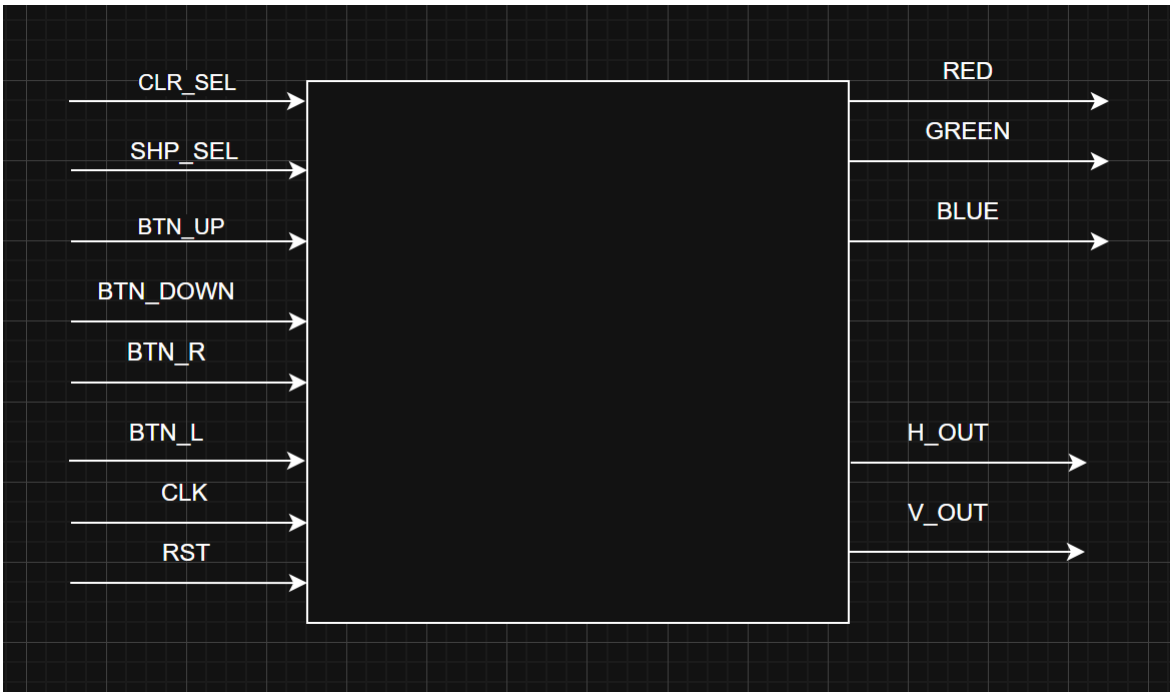
1.TASK

Using a FPGA board implement a VGA controller. 4 different images must be displayed, the selection being read from the board buttons. Images must demonstrate the ability to select colors (at least 4 different colors). Also, the positioning of the images on the screen must be controllable on two axes with the help of the buttons. Image suggestions: square, vertical stripes, horizontal stripes, triangle, circle, etc. The times required for the various resolutions can be found, for example, in Table 2-6 of the XUPV2P_User_Guide.pdf FPGA board documentation, on pages 37-38. Operating diagrams and explanations can be found in the reference manuals of FPGA boards. The project will be carried out by 2 students.



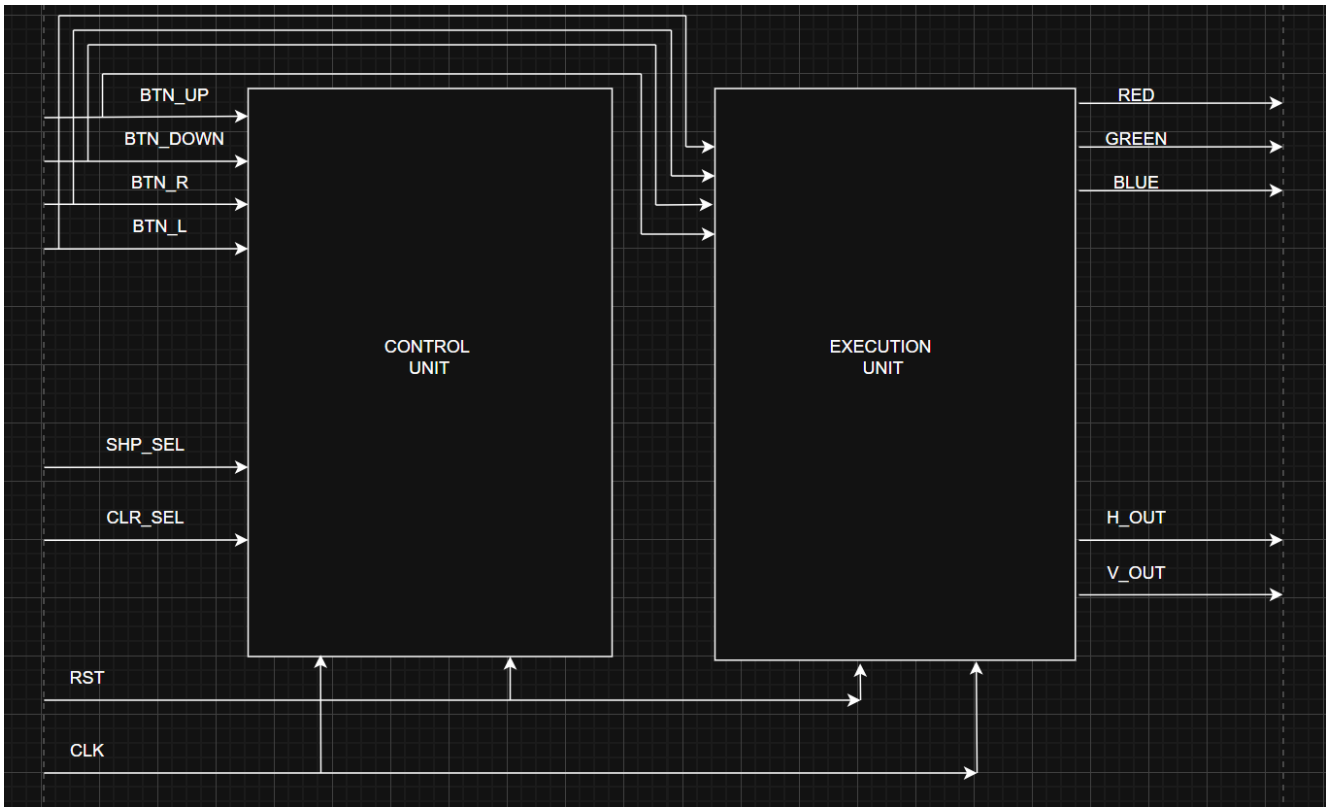
2.DESIGN

a. Black Box



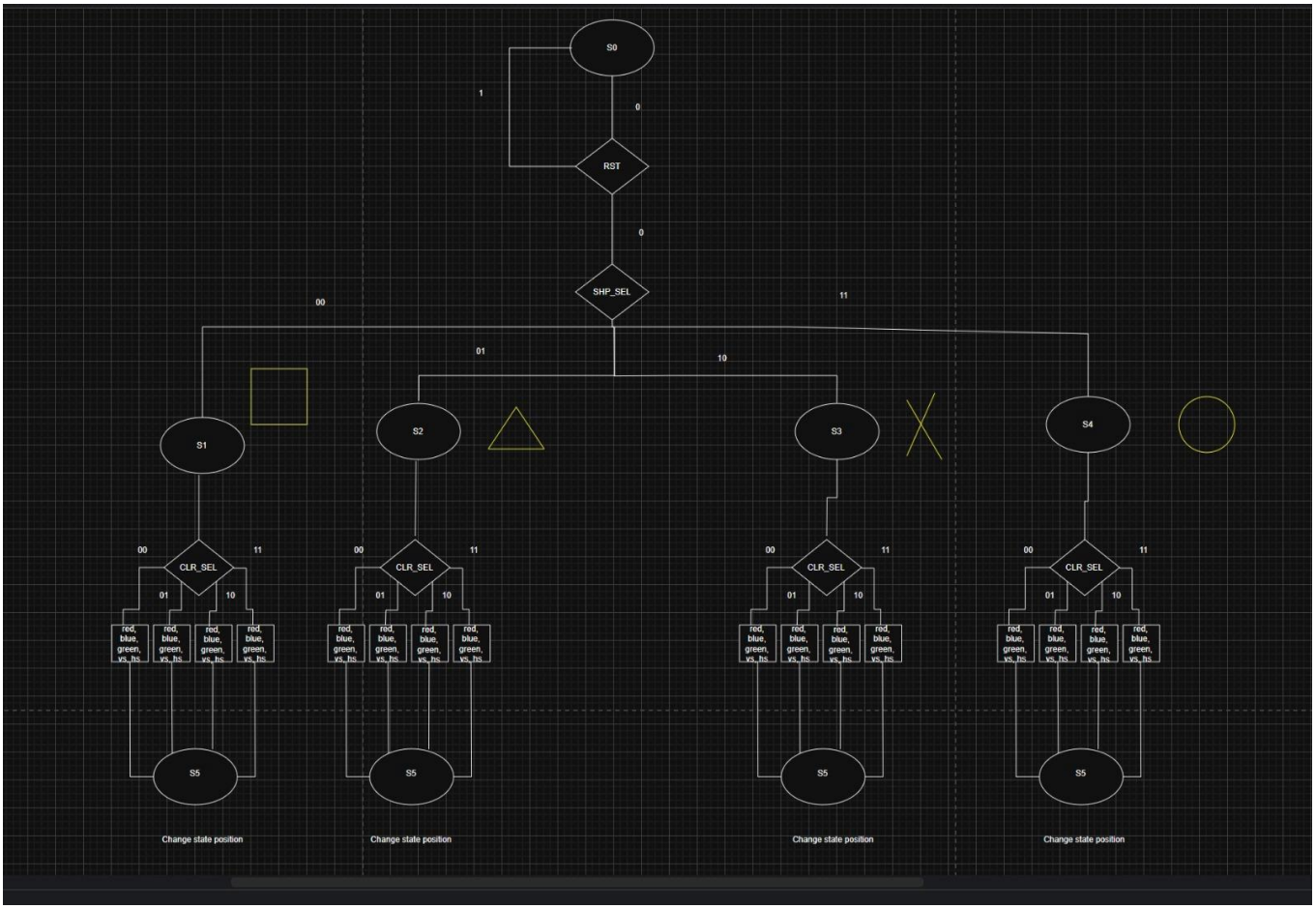
Our project has for inputs: color and shape selection in order to control the aspect of the shapes on the screen, buttons for controlling the direction of movement, a reset switch and a main clock. As for the outputs, we will be able to display on the monitor shapes in different colours using combinations of red, blue and green. Moreover, they will be able to move horizontally and vertically on our command.

b. Control Unit + Execution Unit



This visual representation is highlighting mostly the inputs and outputs (signals will be presented later on).

c. State Diagram

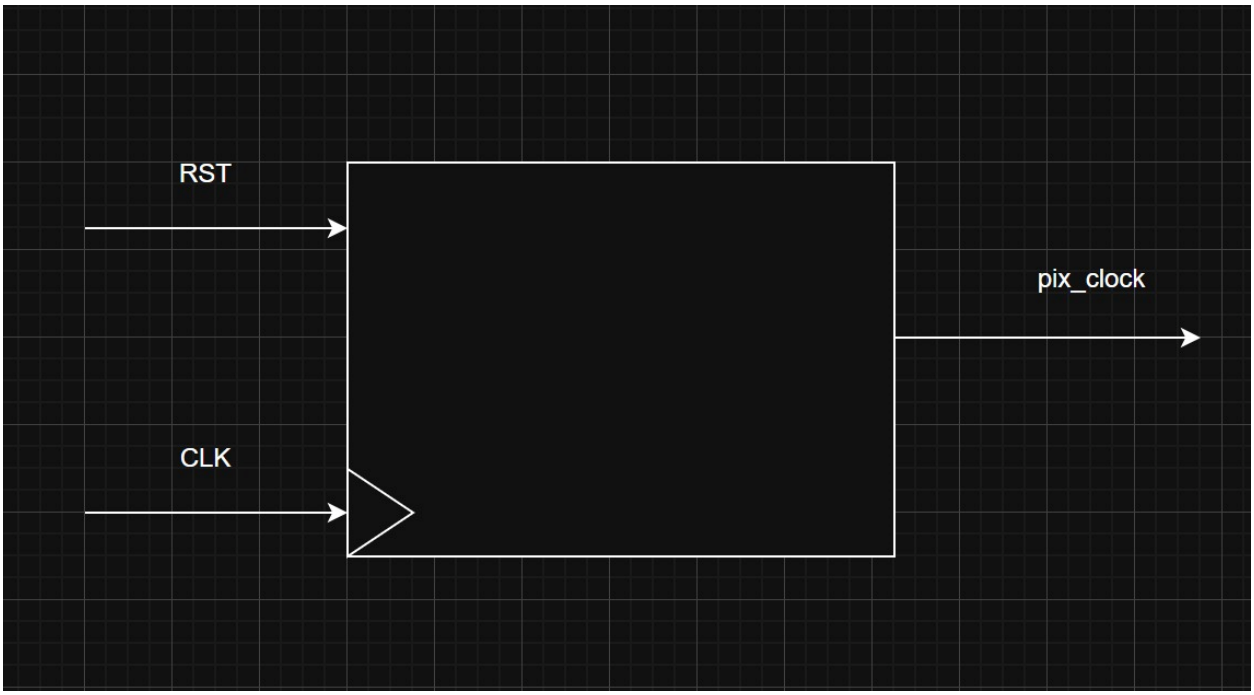


In the State Diagram you can see that, based on the colour and shape selection, we can control the output on the monitor:

-for **shape** with the code 00: we will display a square, whereas for 01 we will have a triangle, for 10 we will draw an X shape and for 11 a circle.

-for **colour** we have a 3-bit vector which is connected with other 3 (RED, BLUE, GREEN – for the intensity of the colours on the monitor), combinations are diverse and it remains for the user to discover all 8 combinations (2^3=8).

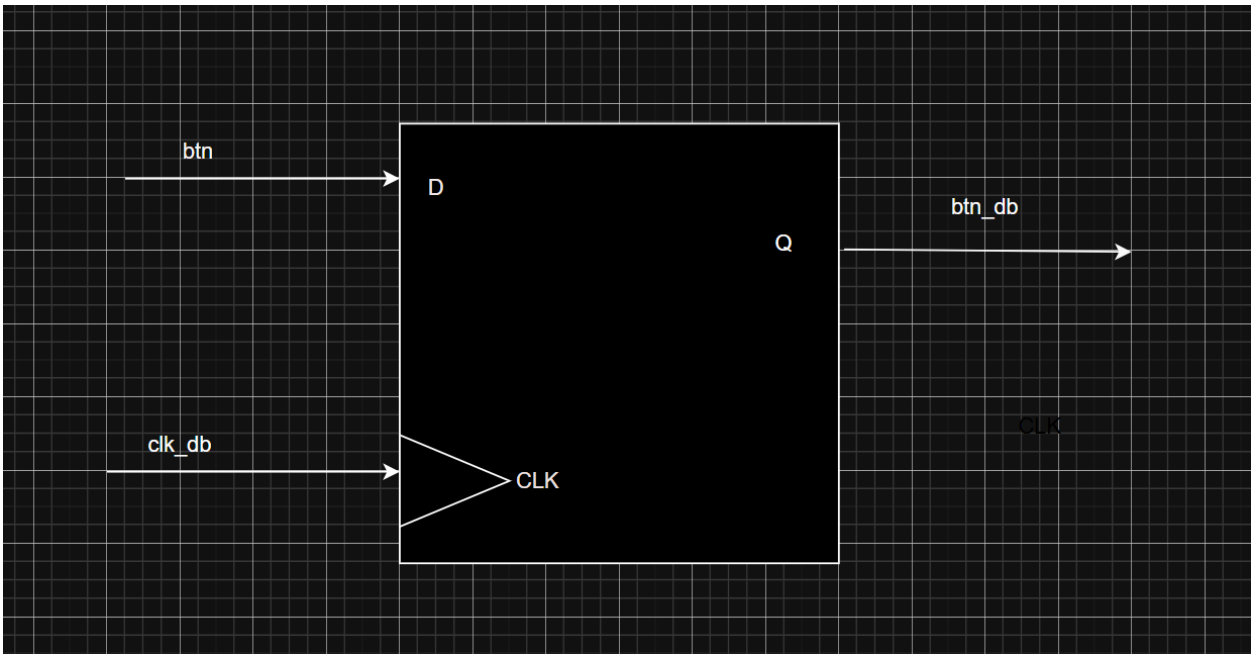
d. Frequency Divider- for pixel clock

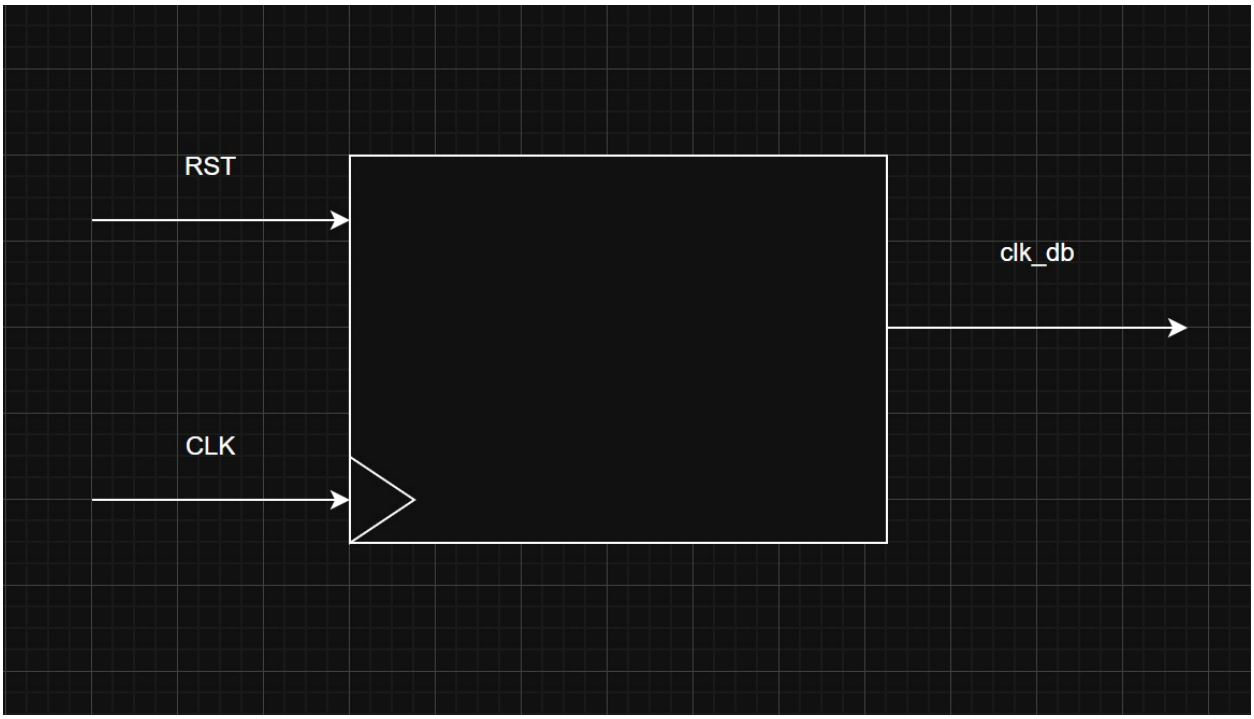


We used a frequency divider in order to divide the 100 MHz into the 25 MHz, according to the monitors' 640x480 resolution.

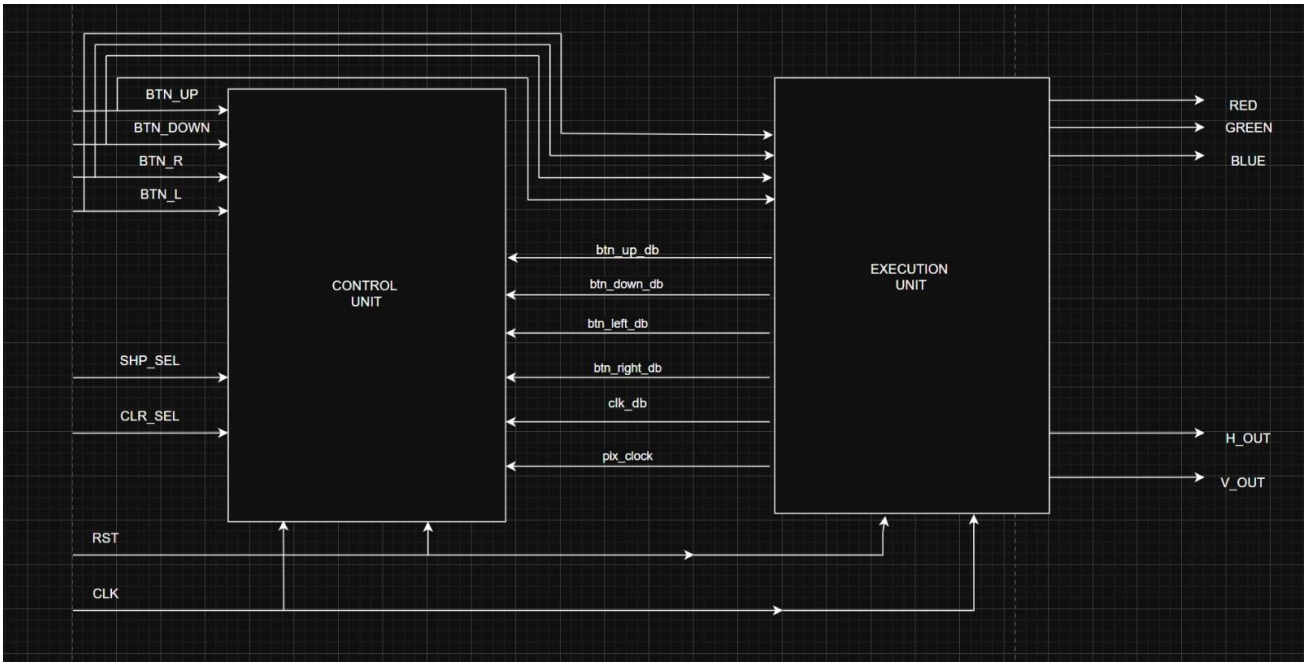
e. Debouncer

The debouncer is made out of one D flip flop and it is used in order to generate a stable signal when switches/buttons are pressed. There was also used a frequency divider for the debouncer's clock.



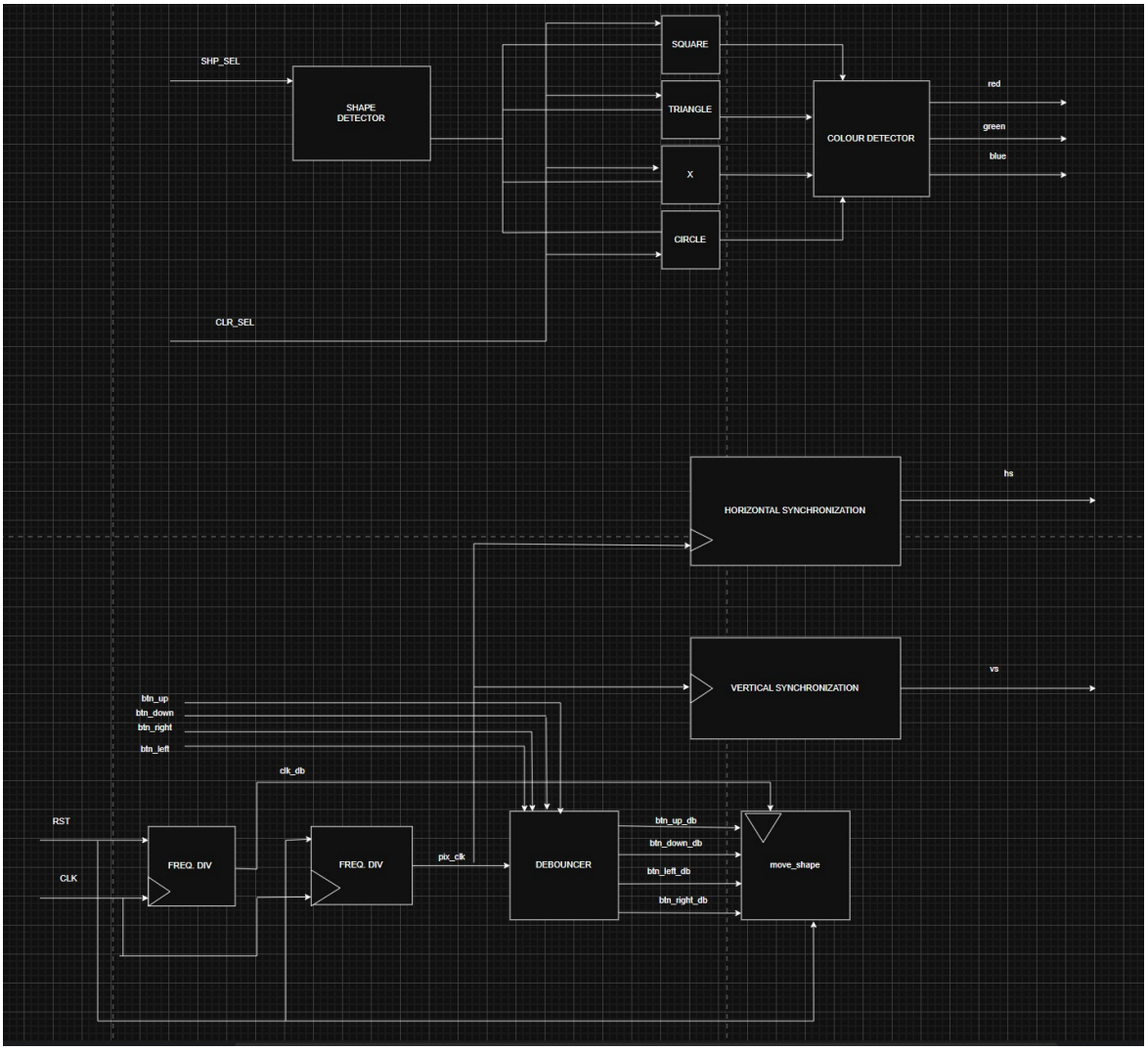


f. Block Diagram for first breakdown



In this picture, there can be seen the signals that we used for the functionality of the project. These mostly focus on the function of moving the shapes up, down, left, right.

g. Detailed Diagram of the Project

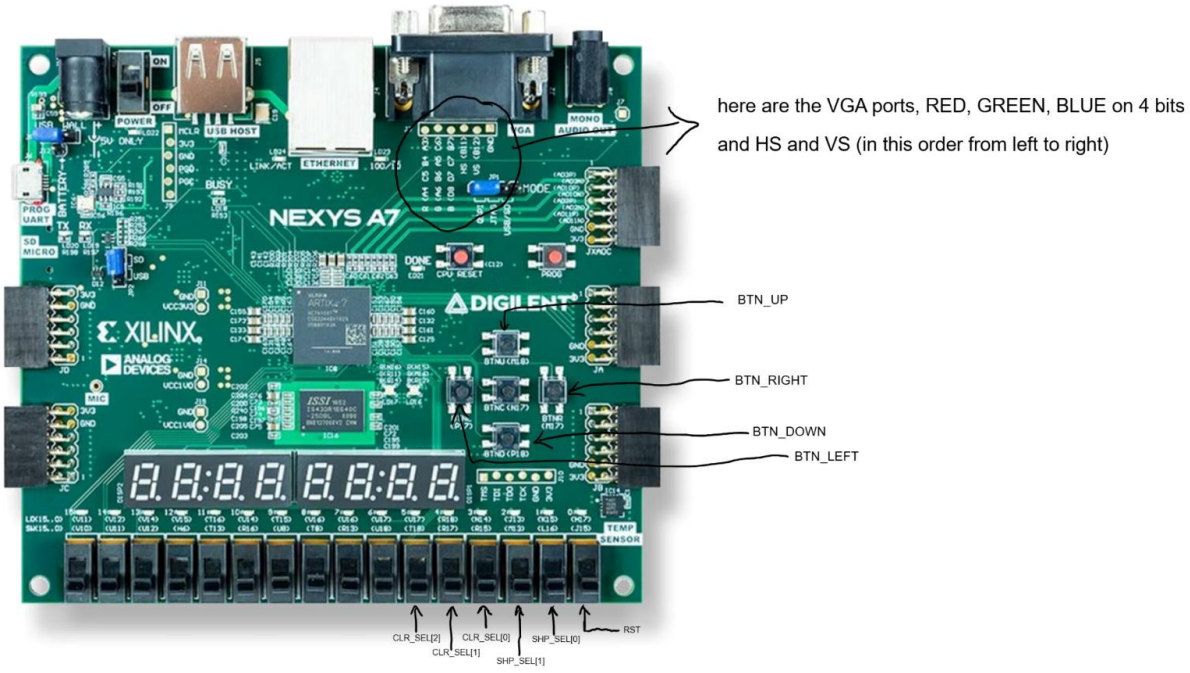


Our project focuses on 2 main functionalities: the selection of colour and shape and the way the user can control their position. In the upper part of the diagram, there can be spotted the way in which we created the first main functionality, whereas in the second half, there are the components that contribute to the effectiveness of the movement of the shapes.

3.USER MANUAL

The project was implemented on a FPGA NEXYS A7, and in order to implement it, a monitor and a VGA cable are needed.

To test its functionalities, the FPGA must be connected to the laptop/computer with the code opened on Vivado and to the monitor through the VGA cable. Moreover, the following buttons/switches were used for its different operations:



Buttons:

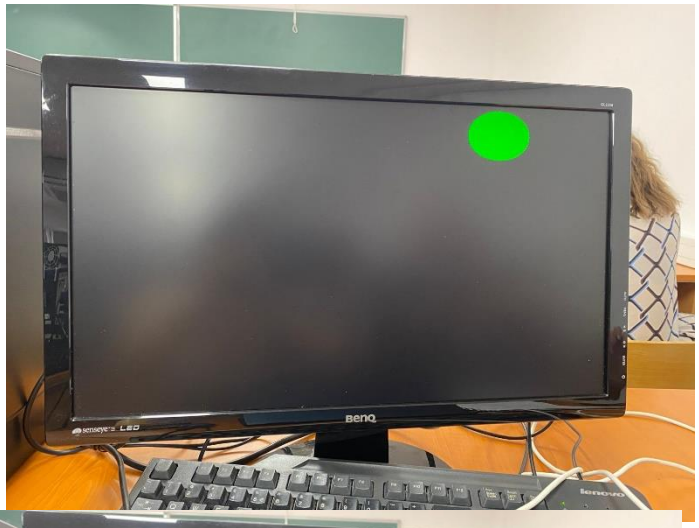
-The **BTNU (M18), BTND (P18), BTNR (M17), BTNL (P17)** –correspond to **BTN_UP, BTN_DOWN, BTN_RIGHT, BTN_LEFT**. They are used to move the shapes on the Ox (BTN_RIGHT and BTN_LEFT) and Oy (BTN_UP and BTN_DOWN) axes.

- Switches:

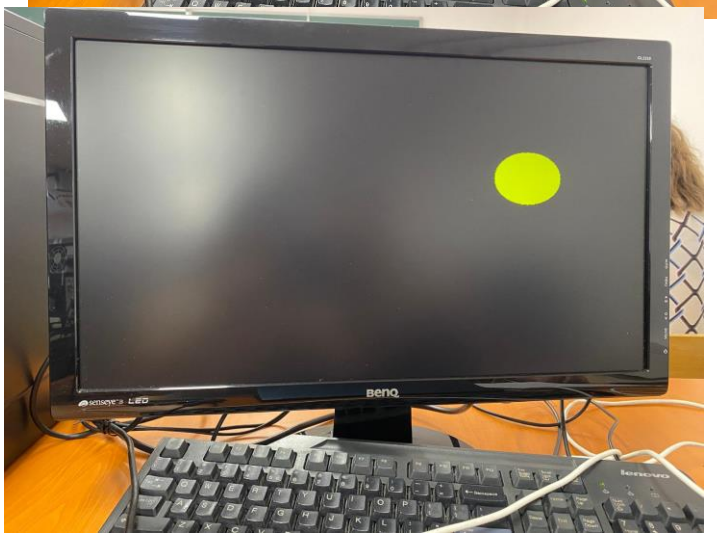
-The **J15 switch** –**RST** serves for resetting the operations. When turned on, it places the shape back in the middle of the screen, no matter the position before.

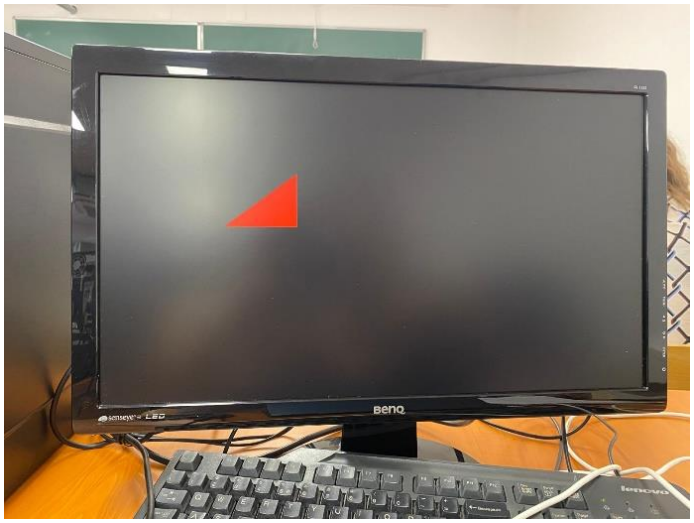
-The next 2 switches**, L16 and M13** occupy with the selection of the shape. L16 represents **SHP_SEL[0]**, whereas M13 is **SHP_SEL[1]**, and each combination of them give a different shape: “00” will give a square, “01” will display a triangle, “10” will select an X shape and “11” will choose a circle.

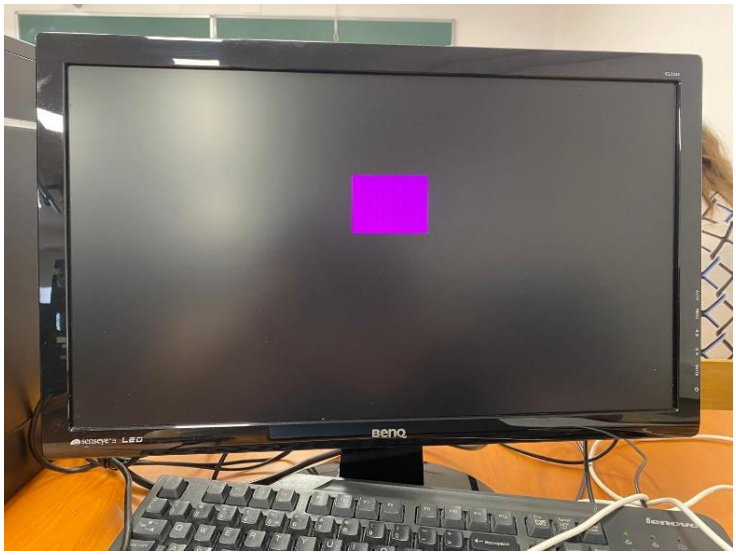
-For the colour selection, we decided to place **CLR_SEL[0]**, **CLR_SEL[1]** and **CLR_SEL[2]** on the switches **R15**, **R17**, and **T18**, in this order. By choosing different combinations of 0s and 1s, the user can easily change the colour of the shape.

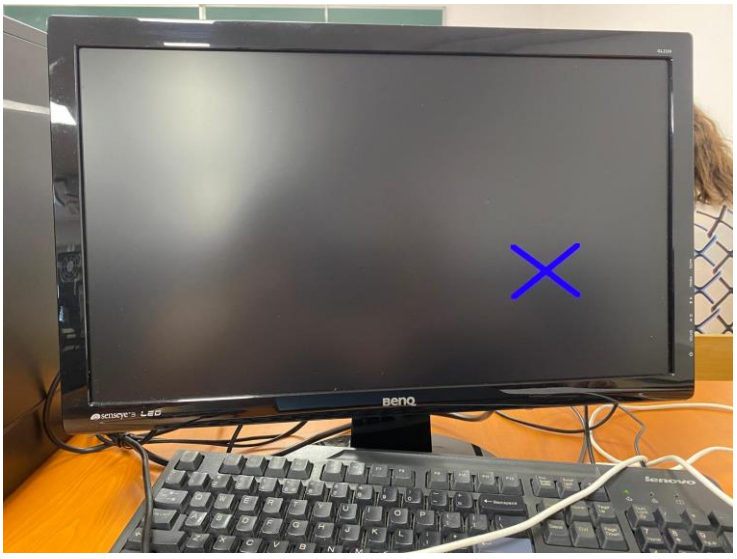














In these pictures, there can be seen all the shapes (**square, triangle, X shape, circle**) and all the colours (**green, turquoise, purple, red, blue, yellow, white, black**-if black then the screen is all black, so we didn't include it) implemented in our project, as well as all the directions in which we can move them: up, down, left, right.

4.Technical Justifications for the Design

The modified clock frequency from 100 MHz to 25 MHz was done for the 640x480 resolution that we used.

General Timing

Screen Refresh Rate	**60 Hz**
Vertical Refresh	**31.47 kHz**
Pixel frequency	**25.175 MHz**

Horizontal Timing (line)

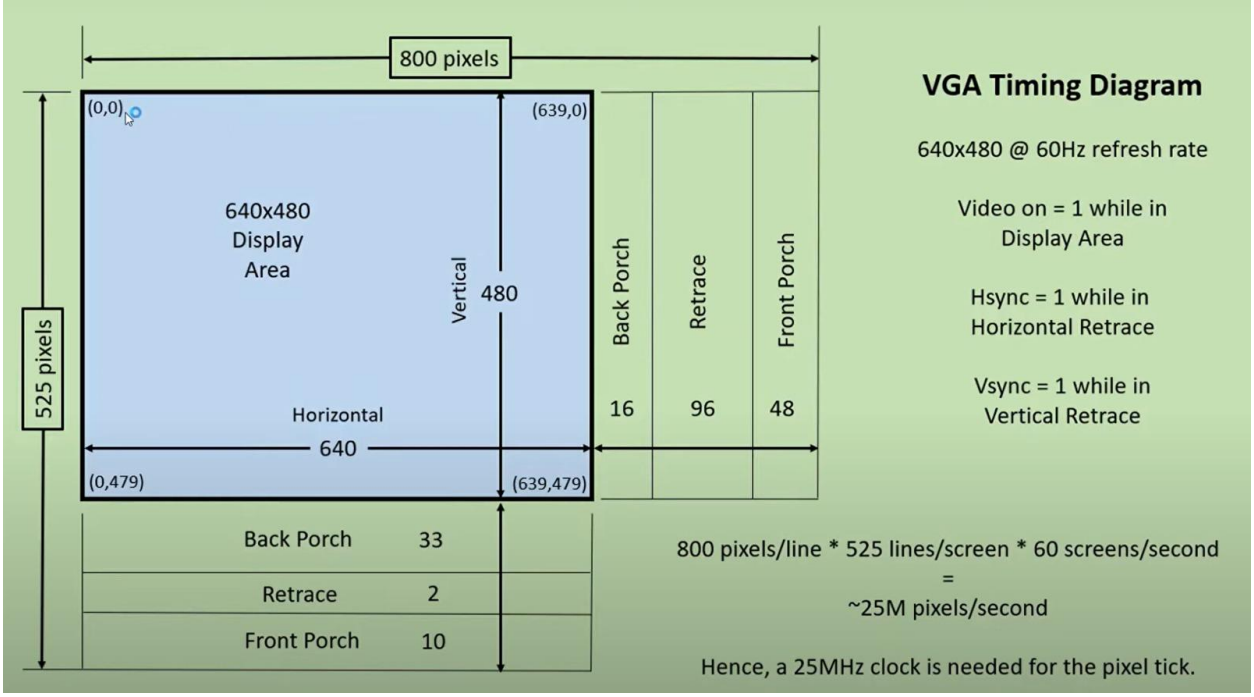
Scanline Part	Pixels	Time(pixels/pixel clock) µs
Visible Area	**640**	**24.42 µs**
Front Porch	**16**	**0.635 µs**
Sync Pulse	**96**	**3.813 µs**
Back Porch	**48**	**1.907 µs**
Whole Line	**800**	**31.78 µs**

Vertical Timing (frame)

Frame Part	Lines	Time(lines/time per line) ms
Visible Area	**480**	**15.254 ms**
Front Porch	**10**	**0.318 ms**
Sync Pulse	**2**	**0.064 ms**
Back Porch	**33**	**1.049 ms**
Whole Frame	**525**	**16.667 ms**

Shapes:

Each shape was drawn by using a mathematical equation that we applied, making use of this picture.



The idea was to create a square shape that will move along the axes and that will enable only some pixels to change colour at a time. In order to change the shapes, we modified the equations so that inside this firstly created square, we will include the triangle, circle and X shape. Therefore, every time we will try to move the shapes, the square will move as well with the shape in it. What doesn't make it obvious is the colour of the pixels which change according to the selection of the shape. For example, when we switch from a square to the triangle, half of the pixels from that square become black, letting the user change the colour only of the "active" pixels which form the triangle. So, it is just like we will "deactivate" (turn to black) or "activate" (enable pixels to change colour) pixels, using mathematical equations.

- Square draw: The pixel is colored only if the coordinates used are placed in the area enclosed by the x, x_offset, y, y_offset integers, where x - x_offset and y – y_offset represent the length of square edge. The coordinates of x, x_offset, y and y_offset are chosen in order to place the square in the middle of the screen when a 640 x 480 resolution is used.

- Triangle draw: The pixel is colored only if the coordinates used are placed in the area enclosed by the x, x_offset, y, y_offset integers (inside the square) and if the coordinates intersection is placed under the secondary diagonal of the square.
- “X” draw: The pixel is colored only if the coordinates used are placed in the area enclosed by the x, x_offset, y, y_offset integers (inside the square) and if the coordinates intersection is placed on either of the 2 diagonals of the square.
- Circle draw: The pixel is colored only if the coordinates used are placed in the area enclosed by the x, x_offset, y, y_offset integers (inside the square) and if the general equation for a circle is fulfilled:

$$(x-a)^2+(y-b)^2=r^2$$

5.Future Developments

Along the facilities that the project has, we considered other possible improvements that would benefit the VGA Controller, such as: implementing more shapes, using mathematical equations (like vertical or horizontal lines or 3D shapes), creating more colours, which will lead to the extension of the number of bits of CLR_SEL and SHP_SEL, allowing the user to place the wanted shape from the beginning in the wanted position. Additionally, we could upgrade the movement of the shapes to also work diagonally.